

Computation for Health Data - Week 3

Vicki Yiannakis

First we will read in the maternal_mortality CSV

```
matmor0 <- read.csv("../data/raw/maternal_mortality.csv", header = TRUE)
glimpse(matmor0)
```

```
Rows: 186
Columns: 22
$ iso      <chr> "AFG", "AGO", "ALB", "AND", "ARE", "ARG", "ARM", "ATG", "AUS~
$ indicator <chr> "Maternal mortality ratio (modeled estimate, per 100,000 liv~
$ X2000    <int> 1450, 827, 23, NA, 6, 66, 43, 44, 7, 6, 47, 1010, 8, 520, 51~
$ X2001    <int> 1390, 766, 23, NA, 6, 67, 42, 44, 6, 6, 44, 956, 8, 516, 501~
$ X2002    <int> 1300, 690, 21, NA, 5, 65, 39, 43, 6, 6, 43, 925, 7, 511, 486~
$ X2003    <int> 1240, 628, 21, NA, 5, 65, 38, 44, 6, 6, 44, 890, 7, 510, 471~
$ X2004    <int> 1180, 574, 18, NA, 5, 61, 36, 41, 5, 5, 39, 844, 7, 505, 454~
$ X2005    <int> 1140, 519, 22, NA, 5, 59, 35, 40, 5, 6, 42, 814, 7, 500, 437~
$ X2006    <int> 1120, 473, 18, NA, 4, 57, 36, 46, 5, 5, 35, 785, 7, 493, 422~
$ X2007    <int> 1090, 431, 19, NA, 4, 56, 32, 48, 5, 5, 34, 756, 7, 486, 410~
$ X2008    <int> 1030, 395, 20, NA, 4, 53, 36, 50, 5, 5, 32, 733, 7, 480, 401~
$ X2009    <int> 993, 359, 20, NA, 4, 56, 32, 45, 5, 5, 32, 698, 6, 471, 393,~
$ X2010    <int> 954, 326, 21, NA, 4, 51, 32, 44, 5, 5, 31, 665, 6, 464, 385,~
$ X2011    <int> 905, 300, 22, NA, 4, 48, 30, 43, 6, 5, 30, 635, 6, 458, 377,~
$ X2012    <int> 858, 281, 17, NA, 4, 47, 30, 44, 6, 5, 29, 608, 6, 450, 369,~
$ X2013    <int> 810, 269, 16, NA, 3, 44, 26, 43, 6, 5, 28, 591, 6, 441, 362,~
$ X2014    <int> 786, 258, 16, NA, 3, 42, 27, 42, 6, 5, 28, 576, 6, 432, 353,~
$ X2015    <int> 701, 251, 15, NA, 3, 41, 28, 43, 6, 5, 27, 568, 5, 421, 343,~
$ X2016    <int> 673, 246, 16, NA, 3, 40, 26, 43, 6, 5, 26, 558, 5, 408, 331,~
$ X2017    <int> 638, 241, 15, NA, 3, 39, 26, 42, 6, 5, 26, 548, 5, 397, 320,~
$ X2018    <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
$ X2019    <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

We want to create a function that uses the below code pivots this to long form

```
# Change wide to long format
matmor_long <- matmor0 |>
  pivot_longer(cols = starts_with("X"), #selecting all cols that start with x
               names_to = "year", #creating new variable called Year
               names_prefix = "X", # removes X from year column (so only numerical shows)
               values_to = "maternal_mortality") |> #values of maternal mortality
  mutate(year = as.numeric(year)) |> # change year to numeric As it was stored as a character var
  select(iso, year, maternal_mortality) #output data
```

NOTES: names_prefix = A regular expression used to remove matching text from the start of each variable name.

To create a function, we need to figure out our arguments

```
long_form <- function(x, varname) {
  x|>
  pivot_longer(cols = starts_with("X"), #selecting all cols that start with x
               names_to = "year", #creating new variable called Year
               names_prefix = "X", # removes X from year column (so only numerical shows)
               values_to = varname)|>
  mutate(year = as.numeric(year)) |>
  select(iso, year, all_of(varname)) #values of maternal mortality
}

#now we can try this in the original mortality data
long_maternal <- long_form(matmor0, "maternal_mortality")
```

Now we will try with the other datasets!

```
neonatal <- read.csv("../data/raw/neonatal_mortality.csv", header = TRUE)
neonatal_long <- long_form(neonatal, "Neonatal_Mortality")

infant <- read.csv("../data/raw/infant_mortality.csv", header=TRUE)
infant_long <- long_form(infant, "Infant_Mortality")

under5 <- read.csv("../data/raw/under5_mortality.csv", header=TRUE)
under5_long <- long_form(under5, "Under5_Mortality")
```

2. Prepare disaster data

```
disaster <- read.csv("../data/raw/disaster.csv", header=TRUE)
disaster1 <- clean_names(disaster)
```

Now we will filter to only include the variables we need Make sure to check variable types!

```
class(disaster1$year)
```

```
[1] "integer"
```

```
class(disaster1$disaster_type)
```

```
[1] "character"
```

```
disaster_filter <- disaster1 %>% filter(year >= 2009 & year <= 2019,  
                                         disaster_type == "Earthquake" |  
                                         disaster_type == "Drought") %>%  
  select(year, iso, disaster_type)  
  
kable(disaster_filter|> slice(1:10))
```

year	iso	disaster_type
2009	CRI	Earthquake
2009	HND	Earthquake
2009	AFG	Earthquake
2009	CHN	Earthquake
2009	IDN	Earthquake
2009	IDN	Earthquake
2009	ITA	Earthquake
2009	JPN	Earthquake
2009	IDN	Earthquake
2009	IDN	Earthquake

Create a dummy variable!

```
disaster_filter2 <- disaster_filter %>%  
  mutate(earthquake = if_else(disaster_type=="Earthquake", 1, 0),  
         drought = if_else(disaster_type=="Drought", 1, 0)) %>%  
  mutate((across(where(is.numeric), as.integer)))  
  
kable(disaster_filter2|> slice(1:10))
```

year	iso	disaster_type	earthquake	drought
2009	CRI	Earthquake	1	0
2009	HND	Earthquake	1	0
2009	AFG	Earthquake	1	0
2009	CHN	Earthquake	1	0
2009	IDN	Earthquake	1	0
2009	IDN	Earthquake	1	0
2009	ITA	Earthquake	1	0
2009	JPN	Earthquake	1	0
2009	IDN	Earthquake	1	0
2009	IDN	Earthquake	1	0

3. Prepare the disaster data file

We are creating a binary variable to ascertain whether they had armed_conflict in that year

```
conflict <- read.csv("C:/Users/iamvi/OneDrive - University of Toronto/Year 2/Computation for Heal

#we will need to create a binary_conflict variable for each year 1999 to 2018 (19 total)
#first we need to filter and sum up the number of conflicts each year

conflict1 <- conflict %>% group_by(iso,year)%>% summarise(total_conflicts=sum(best))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by iso and year.
i Output is grouped by iso.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(iso, year))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
#now we are going to create a binary variable in which year is lagged
```

```
conflict1 <- conflict %>% group_by(iso,year)%>% summarise(total_conflicts=sum(best, na.rm=TRUE))%
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by iso and year.
i Output is grouped by iso.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(iso, year))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

MERGING ALL DATASETS We have: - conflict1 - disaster_filter2 - infant_long - matmor_long - neonatal_long - under5_long They all have the ISO term. Hence they will be merged this way

```
full_dataset_list <- list(matmor_long, neonatal_long, infant_long, under5_long, disaster_filter2,
merged_all <- full_dataset_list %>% reduce(full_join, by = c("iso", "year"))

#save as a CSV file and add to proccessed folder
write.csv(merged_all, "Merged_Data.csv", row.names = FALSE)
```